Attention 加速器 IP 设计报告

25 集成 1 郑建枫

一、完成情况

内容：基础+因果 mask+参数化 S 或 d

状态：输入阶段 → Score 阶段 → 权重阶段 → 累加阶段 → 归一化阶段 → 输出阶段

交付物：可综合 RTL（`rtl/attention_ip.v` 顶层 + 两张查找表）、自检 testbench（`tb/tb_attention_ip.v`）、加分项独立测试（`tb/tb_causal.v`、`tb/tb_param.v`）

软件：ModelSim 10.4、vscode

存在问题：指数查找表 LUT 有问题

do "D:/Attention ip/scripts/run_simple.do"　　← 主设计波形

do "D:/Attention ip/scripts/wave_causal.do"　　← 加分项 A

do "D:/Attention ip/scripts/wave_param.do"　　← 加分项 C

二、公式理解及代码实现

2.1 Score：Q 和 K 做点积

$$\text{score}_j = Q \cdot K_j = \sum Q\,[k] \times K_j[k]$$

代码实现一个 8×8 乘法器，每拍算一项乘积，在 18 位累加器上累加，4 拍凑齐一个 Score。

```
291             ST_DOT: begin
292                 dot_acc <= dot_next;
293                 if (dim == D-1) begin
294                     score_buf[tok] <= dot_next;
295                     if (tok == S-1) begin
296                         state   <= ST_MAX;
297                         max_cnt <= {TW_C{1'b0}};
298                     end
299                     else begin
300                         tok <= tok + 1'b1;
301                         dim <= {DW_C{1'b0}};
302                     end
303                 end
304                 else begin
305                     dim <= dim + 1'b1;
306                 end
307             end
```

2.2 Softmax：先找最大值，再做指数

$$x_{max} = \max(\text{score}_j)$$
$$\text{weight}_j = \exp(\text{score}_j - x_{max})$$

```
ST_MAX: begin
    if (max_cnt == 0) begin
        max_reg <= score_buf[0];
        l_sum   <= {LW{1'b0}};
        for (k = 0; k < 4; k = k + 1) acc_reg[k] <= {AW{1'b0}};
    end
    else if (!max_skip && (score_buf[max_cnt] > max_reg)) begin
        max_reg <= score_buf[max_cnt];
    end
    if (max_cnt == S-1) begin
        state <= ST_EXP;
        tok   <= {TW_C{1'b0}};
    end
    else begin
        max_cnt <= max_cnt + 1'b1;
    end
end
```

求 diff 并查指数表。

指数 LUT

diff	exp(diff)	16 × exp(diff)	四舍五入	exp_lut 里存的值
0	1.000000	16.000	16	16
−1	0.367879	5.886	6	6
−2	0.135335	2.165	2	2
−3	0.049787	0.797	1	1
−4	0.018316	0.293	0	0
≤ −5	≤ 0.006738	≤ 0.108	0	0

2.3 L：所有权重加起来

$$L = \sum weight_j$$

```
    weight_buf[tok] <= w_new;
```

2.4 ACC：用权重对 V 加权求和

$$ACC = \sum weight_j \times V_j$$

4 个乘法器并行，一拍同时更新 4 个 ACC 分量。

```
ST_UPDATE: begin
    acc_reg[0] <= acc_n0;
    if (D > 1) acc_reg[1] <= acc_n1;
    if (D > 2) acc_reg[2] <= acc_n2;
    if (D > 3) acc_reg[3] <= acc_n3;
    if (tok == S-1) begin
        state <= ST_NORM;
        nstep <= 2'd0;
        dim   <= {DW_C{1'b0}};
    end
    else begin
        tok <= tok + 1'b1;
    end
end
```

2.5 O：归一化并四舍五入

$$O = round(ACC/L)$$

```
363             ST_NORM: begin
364                 case (nstep)
365                 2'd0: begin
366                     p_reg <= p_next;
367                     nstep <= 2'd1;
368                 end
369                 2'd1: begin
370                     a0_reg  <= p_reg[PW-1:32];
371                     rem_reg <= rem_c;
372                     nstep   <= 2'd2;
373                 end
374                 default: begin
375                     if (o_sgn > 17'sd127)        out_reg[dim] <= 8'sd127;
376                     else if (o_sgn < -17'sd128)  out_reg[dim] <= -8'sd128;
377                     else                         out_reg[dim] <= o_sgn[7:0];
378                     nstep <= 2'd0;
379                     if (dim == D-1) begin
380                         state   <= ST_OUT;
381                         out_cnt <= {DW_C{1'b0}};
382                         dim     <= {DW_C{1'b0}};
383                     end
384                     else begin
385                         dim <= dim + 1'b1;
386                     end
387                 end
388                 endcase
389             end
```

2.6 位宽设计依据

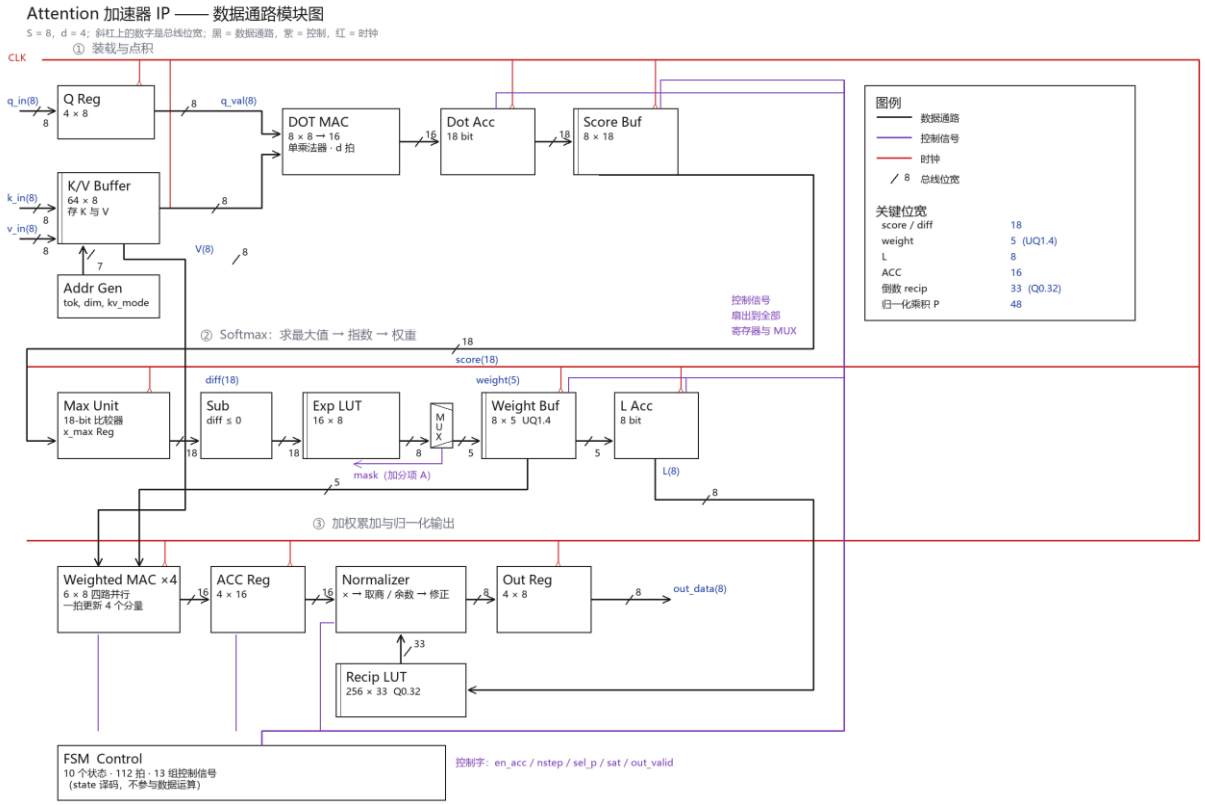
层级	定点格式	位宽
乘积 prod	有符号整数	16
Score / dot_acc	有符号整数	18（SW）
diff	有符号整数	18（SW）
weight / w_new	无符号定点	5（WW）
L / l_sum	无符号定点	8（LW）
ACC / acc_reg	有符号定点	16（AW）
recip_val	无符号定点	33
归一化乘积 P	无符号整数	48（PW）
O / out_reg	有符号整数	8

位宽参数化：上面所有位宽在代码里都写成了 S/d 的函数式，所以换规模时不需要手改任何一行位宽（这就是加分项 C 的基础）：

量	代码里的写法（localparam）	S=8, d=4 时
Score / diff	SW = $clog2(D × 16384) + 2	18
ACC	AW = $clog2(S × 2048) + 2	16
L	LW = $clog2(S × 16) + 1	8
归一化乘积 P	PW = $clog2(S × 2048) + 34	48

三、各模块的设计

3.1 整体架构与数据流



- **存储体**：Q 寄存器、K/V 原始 Buffer、Score Buffer、Weight Buffer、ACC 寄存器、输出寄存器。
- **点积单元**：一个 8×8 有符号乘法器 + 18 bit 累加器，4 拍算出一个 Token 的 Score。
- **权重单元**：减法求 diff → exp_lut → weight，同时把 weight 累加到 L。
- **加权累加单元**：4 个 6×8 有符号乘法器并行，一拍更新 ACC[0..3]。
- **归一化单元**：recip_lut（Q0.32）+ 15×33 乘法器 + 一次余数修正。
- **控制状态机**：10 个状态，负责装载、迭代计数、地址生成与 valid/done 对齐。

图 1 attention_ip 整体架构与数据流

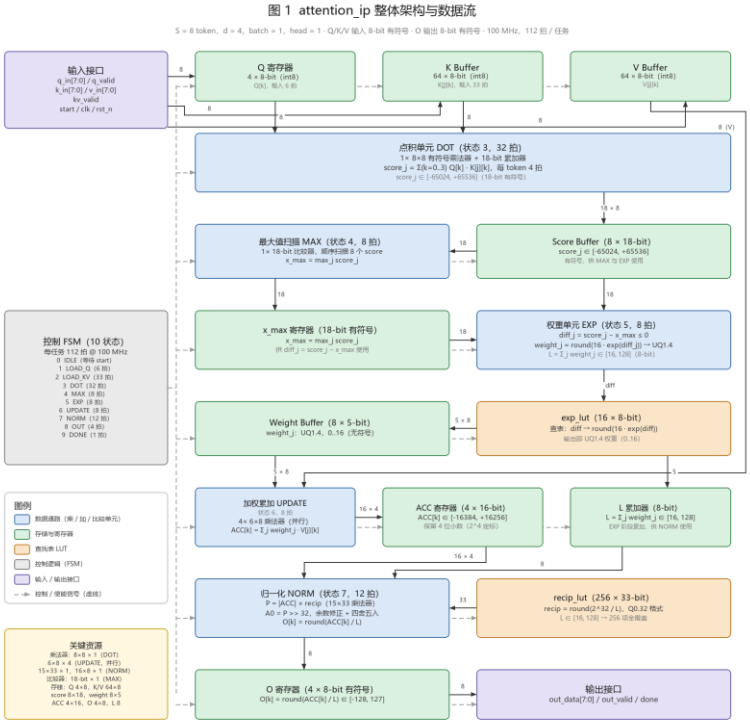


图 1　attention_ip 整体架构与数据流（箭头为数据，虚线为状态机控制）

3.2 状态机与接口时序

状态	名称	拍数	主要动作与使用的资源
0	IDLE	—	等 start；data_ready = 1
1	LOAD_Q	6	收 4 个 q_valid 拍写 Q 寄存器
2	LOAD_KV	33	收 32 拍 K/V 对写 Buffer，并判定输入流格式
3	DOT	32	8×8 乘法器，4 拍累加一个 Score
4	MAX	8	18 bit 比较器顺序求 x_max，同时清零 L 与 ACC
5	EXP	8	diff → exp_lut → weight，累加 L
6	UPDATE	8	4 路并行乘加更新 ACC[0..3]
7	NORM	12	每个输出维度 3 拍：倒数 LUT + 余数修正 + 舍入
8	OUT	4	连续 4 拍输出 O[0..3]
9	DONE	1	done 拉高 1 拍后回 IDLE
	合计	112	

3.3 点积模块 DOT

- **职责**：读 Q 与 K，逐 Token 做 4 项乘加，把 Score 写入 Score Buffer。
- **关键决策：只用一个乘法器串行 4 拍。**面积优先；若改成 4 路并行可压到 1 拍，代价是 4 个 8×8 乘法器（见 5.2 节）。
- **累加器清零放在 dim == 0 的分支里**，用三目运算符实现，不额外占拍。
- **验证**：定向用例 dot_max / dot_min 打到 Score 的上下界，确认 18 bit 位宽是必要的。

3.4 求最大值模块 MAX

- **职责**：在 8 个 Score 中顺序检索最大值，输出 x_max；同时把 L 与 ACC 清零，省掉一个专门状态。
- **关键决策：用 1 个比较器扫 8 拍**，而不是比较树。比较树能 1 拍出结果，但需要 7 个比较器。
- **比较器与减法器都必须是 18 bit 有符号**，与 Score 同宽。
- **加分项 A 的接入点**：max_skip 屏蔽信号在这里生效，使被屏蔽位置不参与求最大值（理由见 3.9 节）。

3.5 指数查表模块 EXP

- **职责**：由 diff 查表得到 weight，同时累加 L。
- **关键决策：地址取 diff 的低 4 位补码**，省掉比较器；代价是必须钳位处理 diff ≤ −16 的绕回（见 4.2 节）。
- **查表与累加同拍完成**，所以 8 个 Token 只要 8 拍。

3.6 加权累加模块 UPDATE

- **职责**：读 Weight Buffer 与 V Buffer，计算 L（在 EXP 已算）并把 8 组 weight × V 累加到 ACC[0..3]。
- **关键决策：4 路并行乘法器。**d = 4 个输出维度天然独立，并行不增加控制复杂度，所以一拍更新 4 个分量，8 拍完成。
- **必须显式扩展符号位**：weight 无符号、V 有符号，直接相乘会按无符号解释（见 4.3 节）。

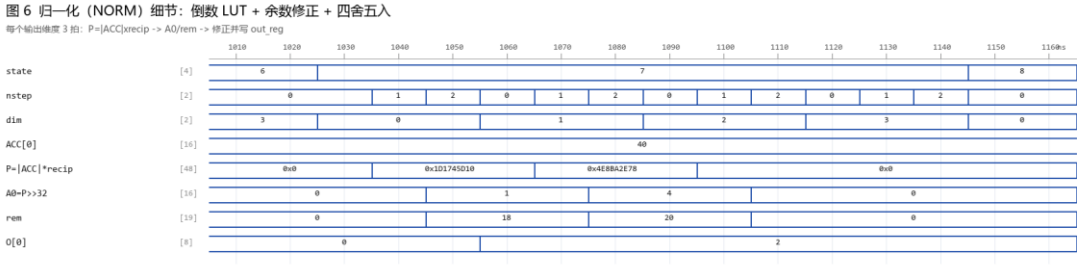


图 6 归一化细节：ACC[0]=40、L=22 时 P=0x1D1745D10 → A0=1、rem=18，2×18 ≥ 22 故进位，O[0] = 2

3.7 加分项 A：因果 Mask

功能：当前 Query 位于位置 q_pos 时，只允许看 j ≤ q_pos 的 Key，j > q_pos 的 Key 被屏蔽（weight 强制为 0）。

实现：

- 新增参数 CAUSAL_EN（默认 0）。CAUSAL_EN = 0 时综合结果与基础版**逐位一致**，不会影响基础验收。
- 新增 4 位输入 q_pos，给出当前 Query 的位置。
- EXP 阶段：causal_block = CAUSAL_EN && (tok > q_pos)，被屏蔽位置的 weight 直接置 0，因此也不贡献 L。
- MAX 阶段：max_skip = CAUSAL_EN && (max_cnt > q_pos)，**求 x_max 时也跳过被屏蔽的位置。**

```
ST_EXP: begin
    weight_buf[tok] <= w_new;
    l_sum           <= l_sum + {{(LW-WW){1'b0}}, w_new};
    if (tok == S-1) begin
        state <= ST_UPDATE;
        tok   <= {TW_C{1'b0}};
    end
    else begin
        tok <= tok + 1'b1;
    end
end
```

3.8 加分项 C：参数化 S / d

实现：把 S、d 提成 parameter（默认 S=8、d=4，即基础题规模），所有位宽、存储深度、计数器宽度都由 S/d 自动推导，改参数不需要手改任何一行位宽。

可配置范围：S ∈ [1, 15]（上限来自 recip_lut 的 8 位地址）

四、遇到的问题

指数 LUT 查找表问题

> 必须先求 x_max，再计算 exp(score− x_max)；指数 LUT 需自行实现；weight 为无符号定点，小数保留 4 位，整数位宽自行推导；0 = round(ACC/L)

保留 4 位小数使用*16（有误差）还是*10000 数据结构大

五、优化方案

加分项 E（指数函数实现方法调研）

调研结论：

方法一：直接查找表（LUT）

原理：将输入范围离散化，预先计算 e^x的值存入 ROM，运行时直接查表。

优点：实现最简单，延迟固定（1-2 周期），精度可控（取决于 LUT 深度）。

缺点：面积随精度指数增长。若输入范围 $x \in [-, 160]$，步长 0.25，则需要 64 个条目；若步长缩小到 0.0625，则需 256 个条目。本题基础版本采用的就是直接 LUT 方案（exp(0)=16, exp(-1)=6, exp(-2)=2, exp(-3)=1）。

方法二：分段线性近似（LUT + 插值）

原理：LUT 只存关键节点，节点之间用线性函数 $e^x \approx a \cdot x + b$逼近。需要一次乘加和少量控制逻辑。

优点：面积比直接 LUT 小，精度可通过增加节点数灵活调整。

缺点：需要乘法器，关键路径增加一个乘法延迟。

典型应用：NN-LUT 等方案采用此方法逼近 Softmax 中的非线性操作。

方法三：多项式近似

原理：用泰勒展开或最小二乘拟合的多项式 $e^x \approx 1 + x + \frac{x^2}{2!} + \cdots$在缩减后的范围上逼近。

优点：无 LUT，纯计算，适合流水线。

缺点：高阶多项式需要多个乘法器；范围缩减需要额外的移位和取模逻辑。

方法四：CORDIC 算法

原理：通过迭代旋转逼近 e^x，利用双曲 CORDIC 模型。相关专利方案将 e^{x_i}转化为 $e^{x_i - Y \ln 2} \cdot 2^Y$，其中 2^Y通过移位实现。

优点：只需移位和加法，无乘法器，适合面积受限场景。

缺点：迭代周期长（通常 10-20 周期），控制逻辑复杂。

方法五：对数域近似（Log-Domain）

原理：将指数、乘法、除法统一转换到对数域，用加法和减法替代。H-FA 方案在 FlashAttention 中对 softmax 归一化和 V 加权部分使用对数域计算，完全消除了指数求值。SOLE 方案的 E2Softmax 同样基于 log2 量化，指数函数仅由移位器和加法器实现，做到 LUT-free 和 multiplication-free。

优点：面积和功耗显著降低（H-FA 报告面积减少 26.5%，功耗减少 23.4%）。

缺点：需要额外的对数域转换硬件；精度受定点格式限制。

方法六：位移-加法近似（Schraudolph 类）

原理：利用 IEEE 浮点位模式与对数的近似关系，直接将整数位移操作映射为 2^x的近似。

优点：极低硬件开销，无 LUT，无乘法。

缺点：精度粗糙，需要校准。